

Project 2: Lossy source coding for image and simple channel coding over BSC and BEC

曾闻天 231880147

1 Experiment Objectives

Project 2 requires the implementation of a lossy source coding for images and simple channel coding based on the Binary Symmetric Channel (BSC) and the Binary Erasure Channel (BEC).

The experimental results provide a comprehensive analysis of system performance from dimensions such as PSNR, pixel accuracy, algorithm complexity, and Bit Error Rate (BER).

2 Experiment Process

2.1 Image Reading and Preprocessing

Function: `load_gray_image(path: Path, max_side: int = 1280) -> np.ndarray`

Use the PIL library to convert the image to grayscale mode. If the long side of the image exceeds `max_side` (default 1280 pixels), scale it proportionally to within that size. LANCZOS resampling is used for scaling to ensure quality.

Finally, return a numpy array of type `uint8` with shape (height, width).

2.2 DCT Transform and Inverse Transform

Function: `dct2(block: np.ndarray) -> np.ndarray`

Function: `idct2(block: np.ndarray) -> np.ndarray`

Implement two-dimensional Discrete Cosine Transform on 8×8 blocks. First, precompute the DCT transform matrix C using the `_dct8_matrix` function, where:

$$C_{k,i} = \begin{cases} \frac{1}{\sqrt{8}} & k = 0 \\ \sqrt{\frac{2}{8}} \cos\left(\frac{\pi k(2i+1)}{16}\right) & k > 0 \end{cases}$$

The forward transform is $C \cdot block \cdot C^T$, and the inverse transform is $C^T \cdot block \cdot C$.

2.3 Quantization Table Generation

Function: `scale_quant_table(quality: int) -> np.ndarray`

Based on the JPEG standard luminance quantization table, dynamically scale according to the quality factor q (1–100). The scaling rule is:

$$\text{scale} = \begin{cases} 5000/q & q < 50 \\ 200 - 2q & q \geq 50 \end{cases}$$
$$Q_{\text{scaled}}(u, v) = \max \left(1, \left\lfloor \frac{\text{JPEG_LUMA}(u, v) \times \text{scale} + 50}{100} \right\rfloor \right)$$

2.4 Zigzag Scanning and Reordering

Function: `block_to_zigzag(qblock: np.ndarray) -> List[int]`

Function: `zigzag_to_block(coeffs: Sequence[int]) -> np.ndarray`

Expand the 8×8 DCT coefficient matrix into a one-dimensional array in Zigzag order. This order prioritizes low-frequency coefficients, causing high-frequency zeros to appear consecutively, which facilitates run-length encoding. The predefined ZIGZAG list stores 64 coordinate pairs (r, c) . Traversal in this order completes the mapping; the inverse transform follows the same principle.

2.5 Run-Length Encoding (RLE) and DPCM

Function: `rle_encode(coeffs: Sequence[int]) -> List[Tuple[int, int]]`

Encode the 64 coefficients after Zigzag (index 0 is DC, indices 1–63 are AC) as follows:

- DC coefficient: Use Differential Pulse Code Modulation (DPCM), encoding the value as $DC_{\text{diff}} = DC_{\text{curr}} - DC_{\text{prev}}$, further compressing using the DC correlation of adjacent blocks.
- AC coefficients: Run-length encoding, encoding the sequence as (number of consecutive zeros, non-zero value) pairs. Encode 16 consecutive zeros as (15, 0), and mark the end of the block with (0, 0) as the EOB (End of Block) marker.

Function: `rle_decode(runs: Sequence[Tuple[int, int]], ac_len: int = 63) -> List[int]`

During decoding, expand according to the run-length pairs. After encountering EOB, pad the remaining positions with zeros.

2.6 Encoding and Decoding

Function: `encode_grayscale(arr: np.ndarray, quality: int) -> bytes`

First, pad the height and width of the image to multiples of 8 using edge replication padding, ensuring the image can be completely divided into 8×8 pixel blocks.

Next, traverse each 8×8 block, subtract 128 from each pixel value to center it around zero, then perform DCT. The transformed coefficients are quantized according to the predefined quantization table and rounded to integers, achieving the core step of lossy compression.

After quantization, perform Zigzag scanning on each block to convert the two-dimensional coefficient matrix into a one-dimensional sequence. Among these, the DC coefficient uses DPCM; the AC coefficients use RLE to record the number of consecutive zero values and the values of non-zero coefficients. After encoding each block, store the resulting DPCM differences and RLE results in a list.

Finally, serialize the original image dimensions, the quality factor used, and the encoded data of all blocks into JSON format, then compress it with zlib at the maximum compression level to obtain the final compressed file.

Function: `decode_grayscale(data: bytes) -> np.ndarray`

Decoding is the inverse process: decompress the JSON, read the DPCM values block by block to restore DC, decode RLE to restore AC, rearrange back into a matrix via Zigzag, multiply by the quantization table, perform IDCT, and finally crop the padded area and clamp to the 0–255 range.

2.7 Bitstream and Byte Stream Conversion

Function: `bytes_to_bits(data: bytes) -> str`

Function: `bits_to_bytes(bits: str) -> bytes`

`bytes_to_bits` converts each byte to an 8-bit binary string and concatenates them. `bits_to_bytes` pads the binary string with zeros to a multiple of 8, then converts every 8 bits into one byte.

2.8 Hamming (7,4) Encoding and Decoding

Function: `hamming74_encode_nibble(d: Sequence[int]) -> List[int]`

Input 4 data bits $[d_0, d_1, d_2, d_3]$ and generate 3 parity bits:

$$p_1 = d_0 \oplus d_1 \oplus d_3$$

$$p_2 = d_0 \oplus d_2 \oplus d_3$$

$$p_3 = d_1 \oplus d_2 \oplus d_3$$

Output codeword $[p_1, p_2, d_0, p_3, d_1, d_2, d_3]$.

Function: `hamming74_decode_codeword(r: List[int]) -> Tuple[List[int], int]`

Calculate the syndrome during decoding:

$$s_1 = p_1 \oplus d_0 \oplus d_1 \oplus d_3$$

$$s_2 = p_2 \oplus d_0 \oplus d_2 \oplus d_3$$

$$s_3 = p_3 \oplus d_1 \oplus d_2 \oplus d_3$$

The binary value of the syndrome (s_1, s_2, s_3) corresponds to the error position (1–7). Flipping that bit completes single-bit error correction.

Function: `hamming_encode_bits(bits: str) -> str`

Function: `hamming_decode_bits(bits: str) -> Tuple[str, int]`

Group the input bitstream into 4-bit groups, encode each group into a 7-bit codeword, and concatenate. During decoding, group into 7-bit codewords, decode each, and return the data bitstream along with the cumulative number of corrections.

2.9 Repetition Code Encoding and Decoding

In this experiment, the number of repetitions per information bit is set to the constant `REP_FACTOR = 10`.

Function: `repetition_encode(bits: str, n: int = 5) -> str`

The logic is: repeat each bit in the input bit string according to the parameter `n` (default 10) to obtain the encoded bit string.

Function: `repetition_decode_hard(bits: str, n: int = 10) -> str`

Suitable for hard-decision decoding in BSC scenarios. Divide the received bit string into groups of `n` bits, count the occurrences of 0 and 1 in each group, and output the bit that appears more frequently as the decision.

Function: `repetition_decode_erasure(bits: str, n: int = 10) -> str`

Suitable for erasure decoding in BEC scenarios. In each group of `n` bits, ignore symbol positions marked as erasures (E), and perform majority voting only on the remaining known bits. If all bits in a group are erased (i.e., no known information is available for decision), default to decoding that bit as 0.

2.10 Channel Models

Function: `bsc_channel(bits: str, p: float, rng: random.Random) -> Tuple[str, int]`

BSC channel: For each bit in the input bit string, flip it with probability p , and keep it unchanged with probability $1 - p$. Returns the output bit string and the number of flips.

Function: `bec_channel(bits: str, epsilon: float, rng: random.Random) -> Tuple[str, int]`

BEC channel: For each bit, output E with probability ε , and output the original bit with probability $1 - \varepsilon$. Returns the output string and the number of erasures.

2.11 Erasure Handling for Hamming Decoding under BEC

Function: `hamming_decode_erasure_bits(bits: str) -> Tuple[str, int]`

When BEC produces erasures, the received bit string may contain E. This function uses an exhaustive search method: for all erasure positions in each Hamming (7,4) codeword, try filling with 0/1 and check whether the three parity equations are satisfied. If a unique valid solution exists, error correction is completed; if no solution exists, default to filling with 0. Returns the decoded bit string and the number of erasure symbols parsed.

2.12 Bitstream Encapsulation and Decapsulation

Function: `pack_bitstream(source_payload: bytes) -> bytes`

Encapsulation format:

[4B magic number "IMG2"] + [4B source payload length] + [4B original bit length] + [Hamming-encoded bitstream (byte-aligned)]

Function: `unpack_bitstream(data: bytes, channel_bits: str, channel: str) -> Tuple[bytes, int]`

Select the corresponding Hamming decoder (normal or erasure version) according to the channel type, then extract the effective bit length, convert back to a byte stream, and return the source payload and the number of corrections.

2.13 End-to-End

Function: `run_single_image(path: Path, channel: str, p_bsc: float, eps_bec: float, quality: int, seed: int) -> PipelineResult`

For a single grayscale image, first load the grayscale image to be processed as the original source. Subsequently, perform lossy source coding on the image and encapsulate the encoding result into a format suitable for transmission, ultimately outputting a bitstream.

In the channel coding stage, first apply Hamming (7,4) coding to the bitstream, then superimpose repetition code (10,1) to enhance error resistance during transmission. The encoded

data is then transmitted through the channel. The channel type can be selected as ideal channel, BSC, or BEC according to experimental requirements to simulate different communication environments.

At the receiving end, first select the corresponding repetition code decoding method according to the channel type: if it is a BSC, use hard-decision decoding; if it is a BEC, use erasure decoding. After completing repetition code decoding, perform Hamming decoding, again selecting normal decoding or a version supporting erasures according to the channel type.

The bitstream recovered after decoding undergoes decapsulation to restore the intermediate format of the lossy source coding, then lossy source decoding is performed, and finally the image is reconstructed. To evaluate the performance of the entire system, after the process is completed, multiple metrics are calculated, including PSNR, pixel accuracy, BER, and the time consumed at each processing stage.

2.14 Performance Evaluation Functions

Function: `psnr(original: np.ndarray, restored: np.ndarray) -> float`

Calculate PSNR:

$$\text{MSE} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (x_{ij} - y_{ij})^2, \quad \text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \text{ dB}$$

Function: `pixel_accuracy(original: np.ndarray, restored: np.ndarray, tol: int = 0) -> float`

Calculate the proportion of pixels where $|x_{ij} - y_{ij}| \leq \text{tol}$. In the experiment, tol is set to 1, allowing a grayscale error of ± 1 .

Function: `bit_error_rate(sent: str, received: str) -> float`

Calculate BER, noting that E symbols in the received string are ignored, comparing only known bits.

2.15 Report Output

Function: `print_result(r: PipelineResult) -> None:`

The output content includes:

Metric	Description
Channel model	Ideal channel, BSC, BEC
Quality factor	40, 75, 90
PSNR	Peak Signal-to-Noise Ratio
Accuracy	Pixel accuracy

BER	Bitstream
Channel error count	Degree of channel corruption
Hamming corrections	Error correction capability
Time consumption	Efficiency measurement

3 Interaction

3.1 Interactive Menu (`interactive_loop`)

Supports three operations:

1. Run complete experiment (IDEAL + BSC + BEC, print report)
2. Single image encoding/decoding (save restored PNG)
3. View the most recent experiment JSON

4 Experiment Results

1. Run `pro2.bat`, select to run the complete experiment on `test1.jpg` and `test2.jpg` in the `images` folder.

2. Program output:

汇总报告 (Project 2 - 有损信源编码 + 信道编码)						
图像	信道	质量	PSNR	准确度	BER	ms
test1	IDEAL	40	33.67	43.21	0.0000	3363.4
test1	IDEAL	75	37.19	55.61	0.0000	5010.0
test1	IDEAL	90	41.56	68.29	0.0000	7702.8
test1	BSC	40	33.67	43.21	0.0000	5684.3
test1	BSC	75	37.19	55.61	0.0000	6667.0
test1	BSC	90	41.56	68.29	0.0000	11071.1
test1	BEC	40	33.67	43.21	0.0000	5029.1
test1	BEC	75	37.19	55.61	0.0000	7914.6
test1	BEC	90	41.56	68.29	0.0000	12955.4
test2	IDEAL	40	31.26	30.97	0.0000	3374.7
test2	IDEAL	75	34.61	39.65	0.0000	5463.5
test2	IDEAL	90	39.14	51.67	0.0000	8557.4
test2	BSC	40	31.26	30.97	0.0000	4539.1
test2	BSC	75	34.61	39.65	0.0000	7542.2
test2	BSC	90	39.14	51.67	0.0001	12471.7
test2	BEC	40	31.26	30.97	0.0000	5457.1
test2	BEC	75	34.61	39.65	0.0000	8917.7
test2	BEC	90	39.14	51.67	0.0000	14662.7
平均 PSNR: 36.24 dB 平均像素准确度: 48.23% 平均耗时: 7576.9 ms						

图 1: test2

3. Result Analysis

In this joint experiment of lossy source coding combined with channel coding, the channel coding scheme using Hamming (7,4) coding superimposed with repetition code (10,1) was applied to transmit two test images under ideal channel, BSC, and BEC respectively.

The experimental results show that the BER under all channel conditions is 0, indicating that the joint error-correcting coding adopted can completely correct the channel errors introduced by BSC and BEC, achieving error-free transmission.

At the same time, as the quality factor increases from 40 to 90, both the PSNR and pixel accuracy of the reconstructed images improve significantly, verifying the rationality of the rate-distortion performance of lossy source coding.

From the perspective of runtime, BEC has the highest computational complexity due to its decoding method of exhaustive filling of erasure bits, and its time consumption significantly exceeds that of BSC and the ideal channel.

Overall, the concatenated scheme of repetition code and Hamming code demonstrates strong error correction capability under moderate channel error rates, but at the cost of

a low code rate and high decoding latency, especially more prominent under the erasure channel.

5 Conclusion

Through the above experiments, the research objectives of Project 2 have been basically achieved.